

面向 LLM / Agent Systems 科研的深度学习零基础课

第 02 讲

Shape、索引、切片、转置与维度变化

本讲目标：从“看懂一个 Tensor 的 Shape”进一步走到“能安全地取出、重排、增加或删除维度”，并理解这些操作在 LLM 中到底改变了什么。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1 为什么这一讲非常重要：深度学习代码经常不是“算法错了”，而是 Shape 错了	3
1.1 本讲学完后必须达到的能力	3
2 复习：Shape 不是装饰，而是结构约束	3
3 Index：索引就是“精确取一个位置”	4
3.1 【原理】一维 Vector 的索引	4
3.2 二维 Matrix 的索引	5
3.3 3D Tensor 的索引：开始连接 LLM	5
4 Slice：切片不是取一个点，而是取一个范围	6
4.1 【原理】Python 切片语法	6
4.2 二维 Matrix 的切片	6
5 在 [B,T,D] 中取数据：必须会的 LLM Shape 练习	7
5.1 取第 0 个样本全部内容	7
5.2 取所有样本的第 1 个 Token	7
5.3 只取最后一个 Token	8
5.4 只保留一个 Token，但保持 sequence 轴	8
6 Reshape：元素不变，重新解释“盒子的形状”	8
6.1 【原理】reshape 做了什么	8
6.2 必要条件：元素总数不能凭空改变	9
6.3 用 -1 让 NumPy 自动推断一个维度	9
7 Reshape 不等于 Transpose	10
7.1 reshape：改变“如何分组”	10
7.2 transpose：交换轴的顺序	10
8 高维 Tensor 的 transpose：为什么 Transformer 经常交换轴	11
9 增加一个维度：expand_dims / unsqueeze 的底层意义	12
9.1 为什么要增加长度为 1 的轴	12
9.2 与 PyTorch 的未来连接	13
10 删除长度为 1 的维度：squeeze	13

11 索引、切片、reshape、transpose：四种操作到底改变什么	14
12 【代码实验】用一个 3D Tensor 把所有操作串起来	14
13 常见误区	15
13.1 误区 1：把 (3,) 当成 (1,3)	15
13.2 误区 2：认为 reshape 会自动“转置”数据	15
13.3 误区 3：索引以后忘记轴已经消失	15
13.4 误区 4：只看 Shape，不看轴的语义	16
13.5 误区 5：随便 squeeze	16
14 与以前 LLM 课程的连接：你以前已经见过这些 Shape 操作，只是还没拆开看	16
14.1 连接 1：Token Embedding	16
14.2 连接 2：Q、K、V	16
14.3 连接 3：最后一个 Token	16
15 科研视角：为什么论文和代码中一定要跟踪 Shape	17
15.1 1. Shape 是复现模型结构的最小检查工具	17
15.2 2. Ablation 有时会改变 Shape，也有时必须保证 Shape 不变	17
15.3 3. Silent Bug 比直接报错更危险	18
16 本讲完整心智模型	18
17 进入下一讲前的自检	18

1 为什么这一讲非常重要：深度学习代码经常不是“算法错了”，而是 Shape 错了

第 01 讲我们已经建立了最基本的结构直觉：一个 Tensor 不只是“一堆数字”，还包含**轴的个数、每个轴的长度，以及每个轴在任务中的语义**。

这一讲开始，我们不再只是观察 Tensor，而是要真正**操作 Tensor**。

在以后阅读 PyTorch、Transformer、LLM 代码时，你会频繁看到类似代码：

```
x = x[:, -1, :]  
x = x.reshape(batch_size, seq_len, hidden_size)  
x = x.transpose(1, 2)  
x = x.unsqueeze(0)  
x = x.squeeze(0)
```

如果只记住这些 API，而没有 Shape 直觉，很容易陷入“代码能跑，但不知道为什么能跑”的状态。

本讲最重要的一句话

对 Tensor 的每一次索引、切片、reshape、transpose、增加维度或删除维度，都应该先回答两个问题：
Shape 会变成什么？每个轴的语义有没有改变？

1.1 本讲学完后必须达到的能力

你需要能做到：

1. 对 1D、2D、3D Tensor 进行索引和切片；
2. 解释为什么普通整数索引可能减少一个维度，而切片通常保留该维度；
3. 理解 reshape 的核心规则：元素总数必须保持一致；
4. 区分 reshape 与 transpose：前者重新解释形状，后者交换轴顺序；
5. 理解 expand_dims / unsqueeze 与 squeeze 的作用；
6. 面对 LLM 中的 [B,T,D] Tensor，能够取出某个样本、某个 Token、某个 hidden feature，并预测结果 Shape；
7. 养成“运行代码前先写 Shape”的习惯。

2 复习：Shape 不是装饰，而是结构约束

假设我们有：

$$X \in \mathbb{R}^{2 \times 3 \times 4}$$

在本讲中，我们把它解释为：

- axis 0: batch, 有 2 个样本;
- axis 1: sequence, 有 3 个 Token;
- axis 2: hidden, 每个 Token 有 4 个特征。

所以:

$$X.shape = (2, 3, 4)$$

我们可以把它想成“两本小册子”，每本有 3 行，每行有 4 个数字。

batch 0: 3 个 Token × 每个 Token 4 个特征

batch 1: 3 个 Token × 每个 Token 4 个特征

整体 Shape: (2, 3, 4)

axis 0 长度 = 2

axis 1 长度 = 3

axis 2 长度 = 4

常见误区 / 风险

同样是 Shape (2, 3, 4)，在别的任务里也可能代表“2 个摄像头 × 3 帧 × 4 个传感器”。因此 Shape 告诉你结构，但轴的语义必须结合任务上下文解释。

3 Index: 索引就是“精确取一个位置”

3.1 【原理】一维 Vector 的索引

假设:

$$x = [10, 20, 30, 40]$$

Python / NumPy 索引从 0 开始:

10	20	30	40
index 0	index 1	index 2	index 3

因此:

```
x[0] # 10
x[2] # 30
x[-1] # 40, 最后一个元素
```

负索引

Python 中 -1 表示最后一个位置，-2 表示倒数第二个位置。这在以后读取“最后一个 Token 的 hidden state”时非常常见。

3.2 二维 Matrix 的索引

设：

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

其 Shape 为 (2,3)。

```
A[0, 0] # 第 0 行, 第 0 列 -> 1
A[0, 2] # 第 0 行, 第 2 列 -> 3
A[1, 1] # 第 1 行, 第 1 列 -> 5
```

数学记号与代码的对应

数学中常写矩阵元素为：

$$a_{ij}$$

表示第 i 行、第 j 列的元素。Python 从 0 开始编号，所以代码里的 `A[1,1]` 是数学教材按 1 开始计数时的“第 2 行第 2 列”。

3.3 3D Tensor 的索引：开始连接 LLM

假设：

$$X.shape = (2, 3, 4)$$

并约定：

[batch, sequence, hidden]

那么：

```
X[0, 1, 2]
```

表示：

第 0 个样本 → 第 1 个 Token → 第 2 个 hidden feature

最终只取出一个数，所以结果是 Scalar，Shape 为 ()。

与 LLM / Transformer / Agent Systems 的连接

以后看到 Transformer hidden state：

$$X \in \mathbb{R}^{B \times T \times D}$$

其中 `X[b,t,d]` 就是在取“第 b 个样本、第 t 个 Token、第 d 个隐藏特征”的一个具体数字。

4 Slice：切片不是取一个点，而是取一个范围

4.1 【原理】Python 切片语法

基本形式：

x[start:stop:step]

其中：

- start：从哪里开始，包含；
- stop：到哪里结束，**不包含**；
- step：步长，默认 1。

例如：

x = np.array([10, 20, 30, 40, 50])

x[1:4] # [20, 30, 40]
x[:3] # [10, 20, 30]
x[2:] # [30, 40, 50]
x[::2] # [10, 30, 50]

初学者最容易犯的错误

x[1:4] 不包含 index 4。Python 的区间通常是“左闭右开”：包含 start，不包含 stop。

4.2 二维 Matrix 的切片

还是：

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

A[0:2, :] # 前两行，所有列
A[:, 1] # 所有行，第 1 列
A[:, 1:2] # 所有行，只取第 1 列，但保留二维结构

这里有一个极重要的区别：

代码	结果含义	Shape
A[:, 1]	精确取第 1 列	(3,)
A[:, 1:2]	切出一个“只有 1 列”的二维块	(3,1)

索引与切片对维度的影响

使用一个整数索引，例如 $A[:, 1]$ ，通常会把那个轴“压掉”；使用切片 $1:2$ ，通常会保留那个轴，只是轴长度变成 1。

这就是为什么：

$$(3,) \neq (3, 1)$$

即使它们都包含 3 个数。

5 在 $[B, T, D]$ 中取数据：必须会的 LLM Shape 练习

设：

$$X.shape = (2, 3, 4)$$

轴语义为：

$$[B, T, D] = [batch, sequence, hidden]$$

下面逐个判断。

5.1 取第 0 个样本全部内容

$X[0]$

原来：

$$(2, 3, 4)$$

batch 轴被整数索引固定为一个具体位置，所以这个轴消失：

$$X[0].shape = (3, 4)$$

含义是：一个样本的 3 个 Token，每个 Token 有 4 个特征。

5.2 取所有样本的第 1 个 Token

$X[:, 1, :]$

batch 轴保留，sequence 轴使用整数索引，因此消失，hidden 轴保留：

$$shape = (2, 4)$$

含义：2 个样本，每个样本只保留第 1 个 Token 的 4 维 hidden vector。

5.3 只取最后一个 Token

```
X[:, -1, :]
```

输出 Shape:

(2, 4)

与 LLM / Transformer / Agent Systems 的连接

很多自回归语言模型推理代码里会关注“最后一个位置”的输出，因为下一个 Token 的预测通常来自当前序列最后位置对应的表示。以后你看到 `x[:, -1, :]`，就应该立刻想到：**所有 batch，只取最后一个 Token，保留全部 hidden features。**

5.4 只保留一个 Token，但保持 sequence 轴

```
X[:, 1:2, :]
```

输出不是 (2, 4)，而是：

(2, 1, 4)

因为 sequence 轴没有被删除，只是长度从 3 变成 1。

6 Reshape：元素不变，重新解释“盒子的形状”

6.1 【原理】reshape 做了什么

假设有：

```
x = np.array([1, 2, 3, 4, 5, 6])
```

它的 Shape 是：

(6,)

我们可以：

```
x.reshape(2, 3)
```

得到：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Shape 变为：

$$(2, 3)$$

也可以：

```
x.reshape(3, 2)
```

变成：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Shape 为：

$$(3, 2)$$

6.2 必要条件：元素总数不能凭空改变

原数组有 6 个数字，因此合法的新 Shape 必须满足：

$$\prod_i s_i = 6$$

例如：

$$(6), (2, 3), (3, 2), (1, 6), (6, 1)$$

都可以。

但：

$$(2, 4)$$

不行，因为需要 8 个元素。

reshape 的基本守恒关系

如果原 Shape 为 $(s_1, s_2, \dots, s_n)$ ，新 Shape 为 $(r_1, r_2, \dots, r_m)$ ，那么必须满足：

$$s_1 s_2 \cdots s_n = r_1 r_2 \cdots r_m$$

也就是：**元素总数守恒**。

6.3 用 -1 让 NumPy 自动推断一个维度

```
x = np.arange(24)
x.reshape(2, 3, -1)
```

前两个维度已经使用：

$$2 \times 3 = 6$$

总共 24 个元素，因此最后一个维度只能是：

$$24/6 = 4$$

所以结果 Shape：

$$(2, 3, 4)$$

常见误区 / 风险

一个 reshape 中最多只能有一个 -1，因为系统只能自动推断一个未知维度。如果写两个 -1，信息不足。

7 Reshape 不等于 Transpose

这是初学者非常容易混淆的一组概念。

7.1 reshape：改变“如何分组”

原矩阵：

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Shape：(2,3)。

如果：

```
A.reshape(3, 2)
```

得到：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

7.2 transpose：交换轴的顺序

如果：

```
A.T
```

或者：

```
A.transpose(1, 0)
```

得到：

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

注意两个结果不同。

操作	核心思想	从 (2,3) 到 (3,2) 后的数值布局
reshape	按原有存储顺序重新分组	[1, 2], [3, 4], [5, 6]
transpose	把“行轴”和“列轴”交换	[1, 4], [2, 5], [3, 6]

一句话区分

reshape 改的是“形状解释”；transpose 改的是“轴的顺序”。虽然二者有时会得到相同的 Shape，但数值排列通常不同。

8 高维 Tensor 的 transpose：为什么 Transformer 经常交换轴

假设：

$$X.shape = (2, 3, 4)$$

我们执行：

```
Y = X.transpose(0, 2, 1)
```

这里不是写“交换第 0 和第 2 个轴”，而是在告诉 NumPy：

- 新 axis 0 ← 原 axis 0
- 新 axis 1 ← 原 axis 2
- 新 axis 2 ← 原 axis 1

因此：

$$(2, 3, 4) \rightarrow (2, 4, 3)$$

把轴当成贴有标签的抽屉

原 Tensor 的轴标签是：

$$[B, T, D]$$

执行 `transpose(0,2,1)` 后，轴标签变成：

$$[B, D, T]$$

数字本身没有丢失，只是组织数字的轴顺序变了。

与 LLM / Transformer / Agent Systems 的连接

以后学习 Multi-Head Attention 时，经常会把 Tensor 从类似：

$$[B, T, H, D_h]$$

转换成：

$$[B, H, T, D_h]$$

目的不是“随便换 Shape”，而是为了让 Head 这个轴来到适合批量矩阵运算的位置。本讲的 `transpose` 就是那个操作的数学和编程基础。

9 增加一个维度：`expand_dims` / `unsqueeze` 的底层意义

9.1 为什么要增加长度为 1 的轴

假设一个样本：

$$x.shape = (4,)$$

它表示一个长度为 4 的向量。

但某个模型接口要求输入具有 batch 维度：

$$[B, D]$$

即使只有一个样本，也应该是：

$$(1, 4)$$

此时可以：

```
x_batch = np.expand_dims(x, axis=0)
```

得到：

$$(4,) \rightarrow (1, 4)$$

如果：

```
x_col = np.expand_dims(x, axis=1)
```

则：

$$(4,) \rightarrow (4,1)$$

核心概念

增加一个长度为 1 的轴不会增加新的数值，它改变的是**结构和语义**。 $(4,)$ 、 $(1,4)$ 、 $(4,1)$ 都有 4 个元素，但后续矩阵运算含义可能不同。

9.2 与 PyTorch 的未来连接

NumPy：

```
np.expand_dims(x, axis=0)
```

以后 PyTorch 中你会写：

```
x.unsqueeze(0)
```

二者思想完全一样。

10 删除长度为 1 的维度：squeeze

如果：

$$X.shape = (1, 3, 4)$$

执行：

```
Y = np.squeeze(X, axis=0)
```

得到：

$$Y.shape = (3, 4)$$

因为 axis 0 长度为 1，可以去掉。

squeeze 不能删除长度大于 1 的轴

如果 $X=(2,3,4)$ ，不能直接要求 `squeeze(axis=0)`，因为 axis 0 有 2 个实际位置。删除它会丢失结构信息，而不是简单去掉“空壳”。

如果写：

```
np.squeeze(X)
```

NumPy 会删除所有长度为 1 的轴。这虽然方便，但科研代码中如果你不确定 Shape，可能产生意外。

例如：

$$(1, 3, 1, 4) \rightarrow (3, 4)$$

科研视角

在复现实验时，“无参数 squeeze”有时会导致 batch size = 1 时出现与 batch size > 1 不同的 Shape。调试时应该非常警惕这种“只在边界样本数量下触发”的维度错误。

11 索引、切片、reshape、transpose：四种操作到底改变什么

操作	是否可能改变 ndim	是否改变元素总 数	核心作用
整数索引	是，通常减少	取子集，所以可能 减少	固定某个轴的一个具体位置
切片	通常保留	取子集，所以可能 减少	取某个轴的一段范围
reshape	可以	不改变	重新组织 Shape，要求 size 守恒
transpose	不改变	不改变	改变轴的排列顺序
expand_dims	增加 1	不改变	加一个长度为 1 的轴
squeeze	减少	不改变	删除长度为 1 的轴

12 【代码实验】用一个 3D Tensor 把所有操作串起来

本讲实验脚本使用：

```
X = np.arange(24).reshape(2, 3, 4)
```

其完整数值为：

$$X = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}, \begin{bmatrix} 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 \end{bmatrix}$$

并约定轴语义为：

$$[B, T, D] = (2, 3, 4)$$

然后逐个验证：

```
X[0]          # -> (3, 4)
X[:, 1, :]    # -> (2, 4)
```

```
X[:, 1:2, :] # -> (2, 1, 4)
X[:, -1, :] # -> (2, 4)
X.reshape(6, 4) # -> (6, 4)
X.transpose(0, 2, 1) # -> (2, 4, 3)
np.expand_dims(X[0], 0) # (3, 4) -> (1, 3, 4)
```

实验前必须先预测

不要先运行再看答案。请先把每一行的输出 Shape 写在纸上, 然后再运行脚本。这里训练的是“Shape 推理能力”, 不是键盘操作能力。

13 常见误区

13.1 误区 1: 把 (3,) 当成 (1,3)

它们元素数一样, 但维度不同:

$$(3,) \quad ndim = 1$$

$$(1, 3) \quad ndim = 2$$

后续矩阵乘法、广播规则和模型接口可能不同。

13.2 误区 2: 认为 reshape 会自动“转置”数据

不会。reshape 和 transpose 可能得到同样的 Shape, 但数字位置关系不同。

13.3 误区 3: 索引以后忘记轴已经消失

例如:

$$X.shape = (8, 128, 768)$$

$X[0]$ 后不是 $(1, 128, 768)$, 而通常是:

$$(128, 768)$$

如果想保留 batch 轴, 可以使用:

```
X[0:1]
```

得到:

$$(1, 128, 768)$$

13.4 误区 4：只看 Shape，不看轴的语义

假设：

$$A.shape = (32, 128, 768)$$

它可以是 $[B, T, D]$ ，也可以在别的模型中代表其他含义。操作之前必须确认轴标签。

13.5 误区 5：随便 squeeze

如果模型在某些情况下 batch size = 1，`squeeze()` 可能把 batch 轴意外删除，导致后续代码只在小 batch 时崩溃或产生静默错误。

14 与以前 LLM 课程的连接：你以前已经见过这些 Shape 操作，只是还没拆开来

14.1 连接 1：Token Embedding

以前你见过：

$$[B, T] \rightarrow [B, T, D]$$

从 Token ID 到 Embedding 后，新增了 hidden / embedding 轴。现在你应该知道 $[B, T, D]$ 不是抽象符号，而是 3D Tensor 的三个轴。

14.2 连接 2：Q、K、V

以后会学：

$$Q = XW_Q, K = XW_K, V = XW_V$$

在进入 Multi-Head Attention 后，还需要把一个 hidden 维度拆成：

$$H \times D_h$$

例如：

$$[B, T, D] \rightarrow [B, T, H, D_h]$$

然后交换轴：

$$[B, T, H, D_h] \rightarrow [B, H, T, D_h]$$

这里使用的正是本讲的 reshape + transpose 思想。

14.3 连接 3：最后一个 Token

自回归生成时常常关注：

```
logits[:, -1, :]
```

如果 logits Shape 是：

$$[B, T, V]$$

那么结果是：

$$[B, V]$$

含义：每个 batch 只取最后一个序列位置，对整个 vocabulary 的 logits。

与 LLM / Transformer / Agent Systems 的连接

这里的 V 是 vocabulary size。以后我们学 Softmax 和 Cross Entropy 时会重新回到这个 Shape: $[B, V]$ 上的 logits 会变成下一个 Token 的概率分布。

15 科研视角：为什么论文和代码中一定要跟踪 Shape

15.1 1. Shape 是复现模型结构的最小检查工具

当你复现论文时，如果模型输出异常，最便宜的第一步通常不是重写算法，而是在关键位置打印：

```
print("after embedding:", x.shape)
print("after attention:", x.shape)
print("after pooling:", x.shape)
```

如果论文预期：

$$[B, T, D]$$

而你的代码突然得到：

$$[B, D, T]$$

即使元素总数完全一样，后续计算也可能已经错了。

15.2 2. Ablation 有时会改变 Shape，也有时必须保证 Shape 不变

如果论文把一个模块替换掉做 Ablation，通常需要保证前后接口兼容。例如原模块输出：

$$[B, T, D]$$

新模块也应输出相同结构，否则性能变化可能混入“模型尺寸 / 接口结构变化”的影响。

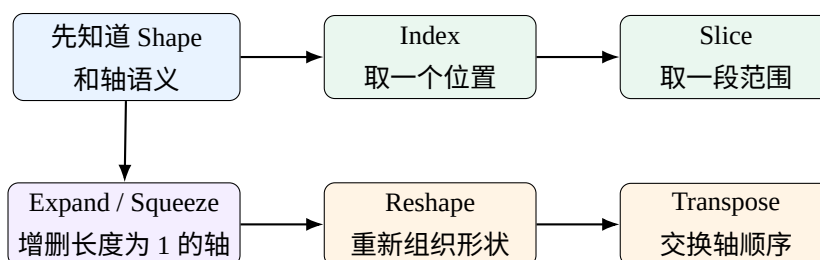
15.3 3. Silent Bug 比直接报错更危险

Shape 完全不匹配时，框架经常会报错，这是好事。更危险的是某些 Shape 虽然不符合你的语义，但因为广播或元素数恰好兼容，代码仍然能跑。

科研视角

以后做 Agent Systems 实验时，你可能不训练基础模型，但仍会处理 embedding、hidden states、scores、batch evaluation outputs。能独立检查 Shape，是判断实验实现是否可信的最低模型基础之一。

16 本讲完整心智模型



本讲最终结论

以后看到任何 Tensor 操作，不要先记 API，而是先画轴。先写出操作前 Shape，再判断哪个轴被固定、保留、拆分、合并或交换，最后写出操作后 Shape。

17 进入下一讲前的自检

如果下面问题能不看答案解释清楚，就可以进入第 03 讲：

1. $(3,)$ 与 $(1,3)$ 有什么区别？
2. 为什么 $X[:,1,:]$ 和 $X[:,1:2,:]$ 的 Shape 不同？
3. `reshape(2,3)` 为什么要求原数组必须正好有 6 个元素？
4. `reshape` 和 `transpose` 的核心区别是什么？
5. 如果 $X=(8,128,768)$ ， $X[:, -1, :]$ 的 Shape 是什么，语义是什么？
6. 为什么 Multi-Head Attention 里经常需要 `reshape` 和 `transpose`？

本节检查点

最低通过标准：给你一个 (B,T,D) Tensor 和一行索引 / `reshape` / `transpose` 代码，你能够在运行之前判断结果 Shape，并用自然语言解释每个轴现在代表什么。